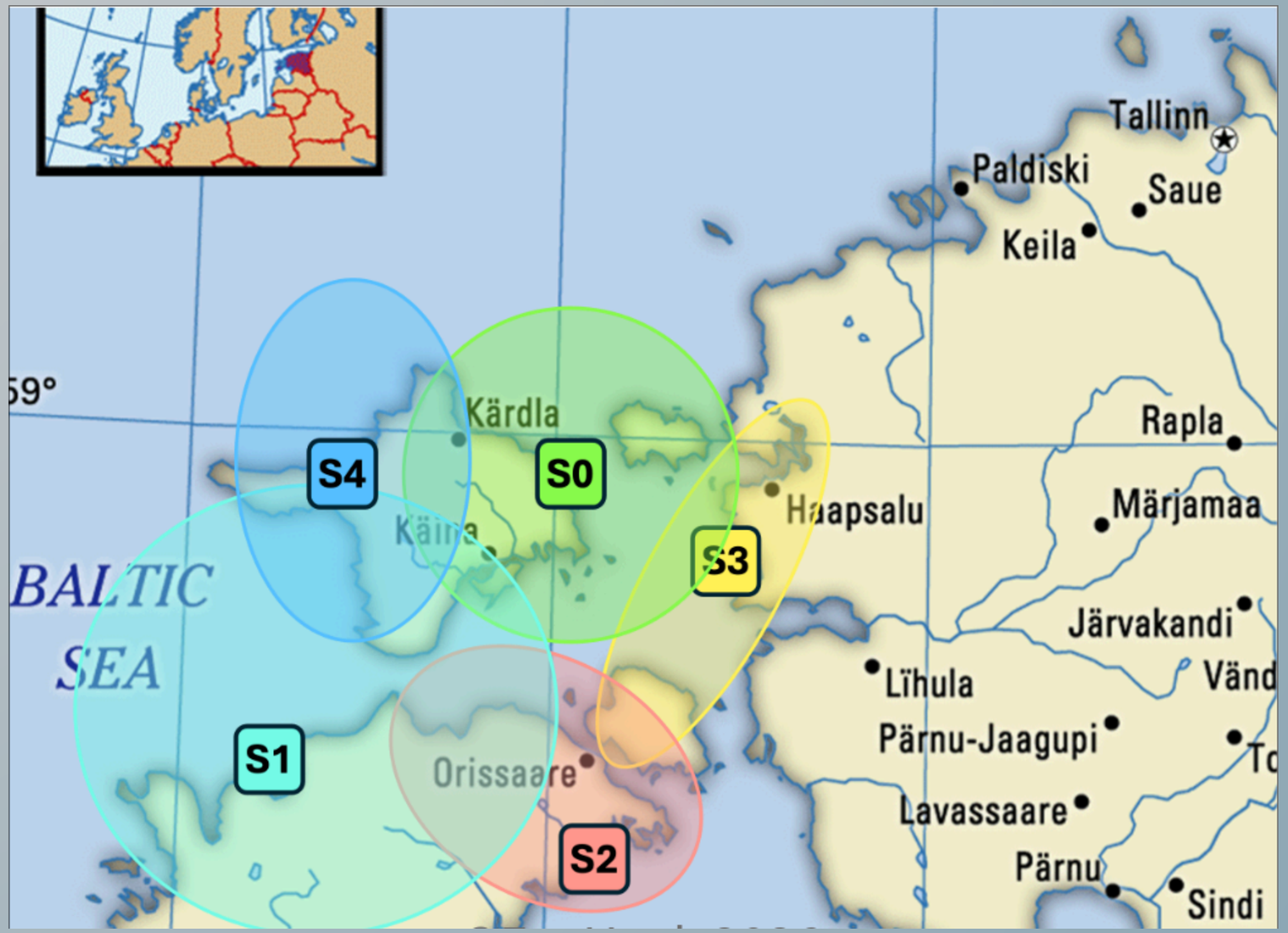
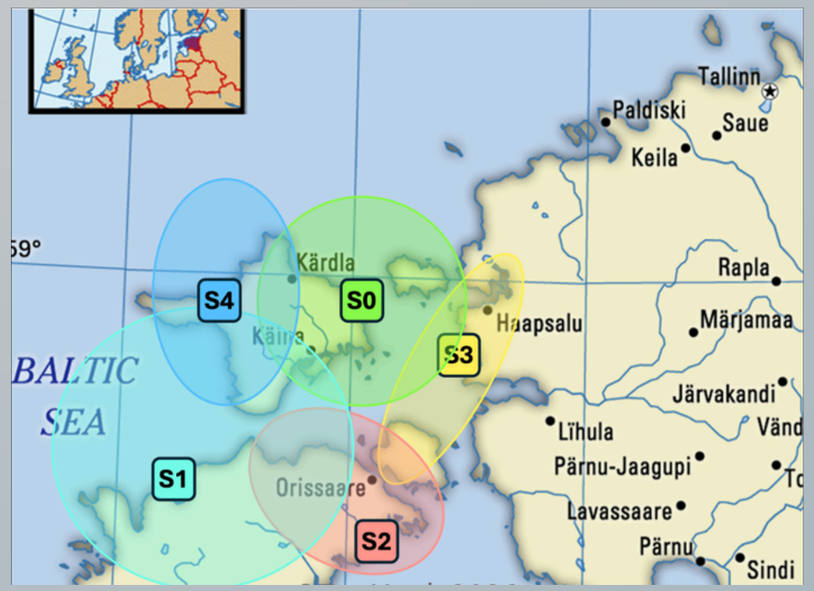
Coop Pank

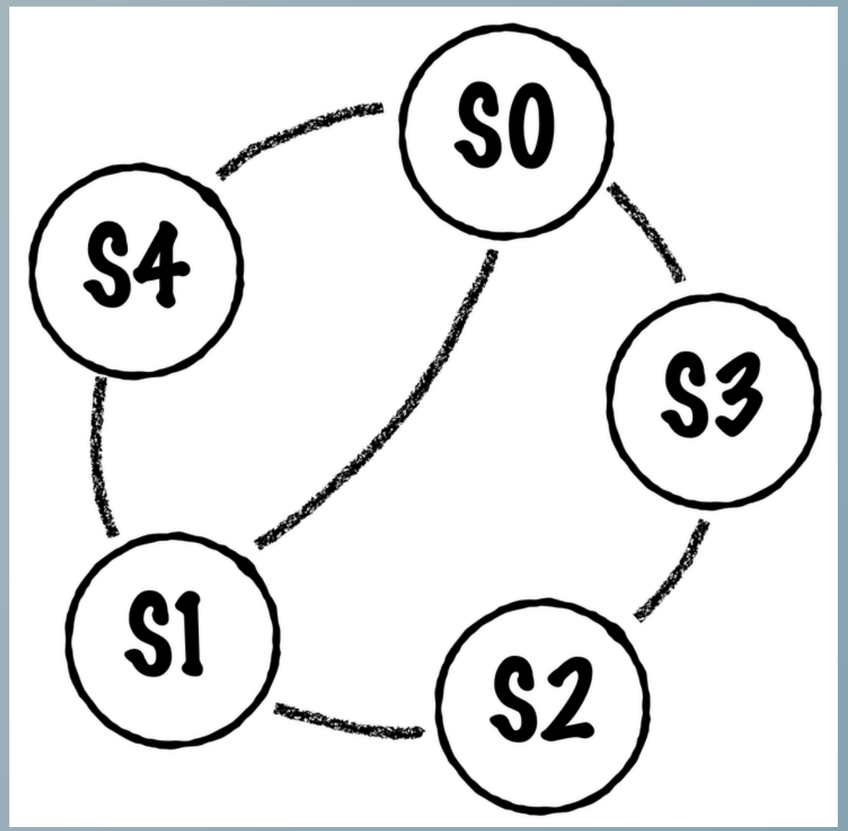
Nikita, Shams, Michal

Problem 1


Tallinn
Paldiski
Saue
Keila
Kärdla
Rapla
S4
S0
Haapsalu
Märjamaa
S3
BALTIC
SEA
Järvakandi
Lihula
Pärnu-Jaagupi
S1
Orissaare
Lavassaare
S2
Pärnu
Sindi

Problem 1

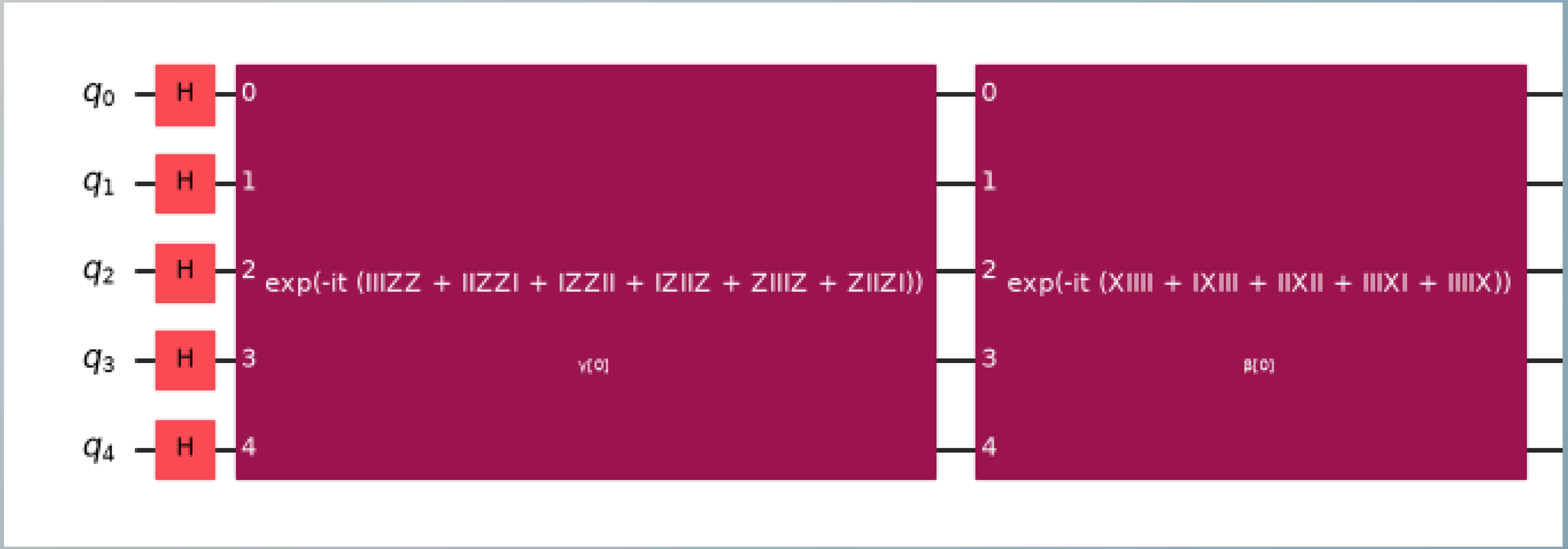

Tallinn
Paldiski
Saue
Keila
Kärdla
Rapla
Haapsalu
Märjamaa
Käina
Järvakandi
BALTIC
SEA
Lihula
Pärnu-Jaagupi
Orissaare
Lavassaare
Pärnu
Sindi
S0
S1
S2
S3
S4


S0
S1
S2
S3
S4

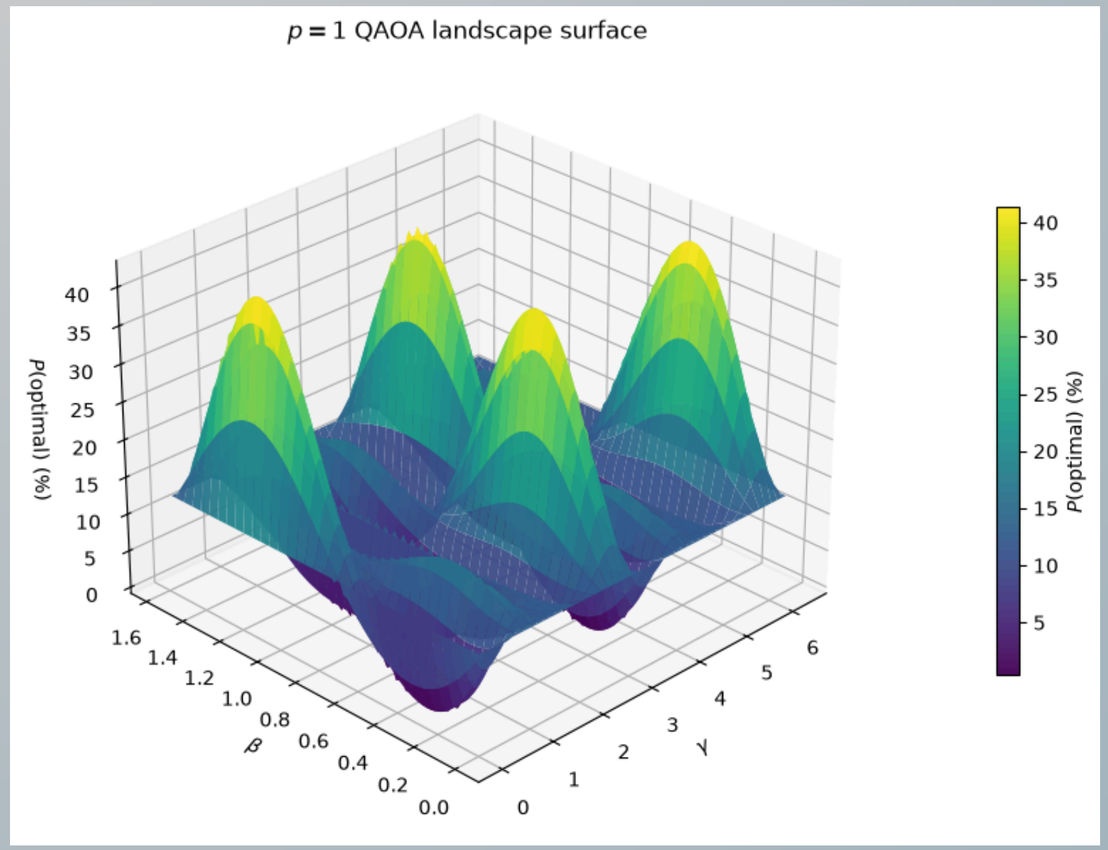
Problem 1

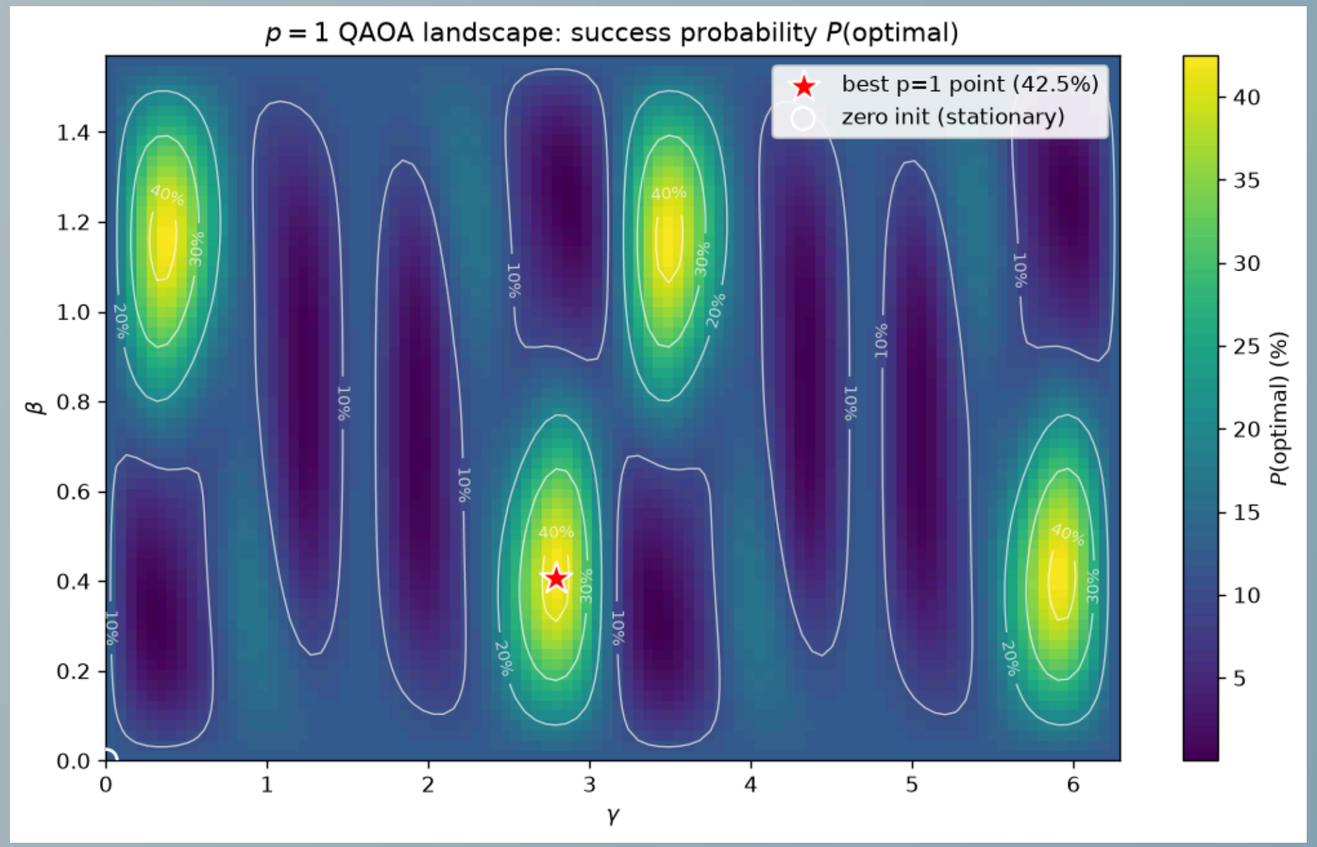
```
# Instantiate the SparsePauliOp as a sum of Pauli chains multiplied by a certain coefficient
H = SparsePauliOp.from_list(
    [("IIIZZ", 1.0), ("IIZZI", 1.0), ("IZZII", 1.0), ("IZIIZ", 1.0), ("ZIIIZ", 1.0), ("ZIIZI", 1.0)]
)
# Display the SparsePauliOp
print(H)
```

Problem 1

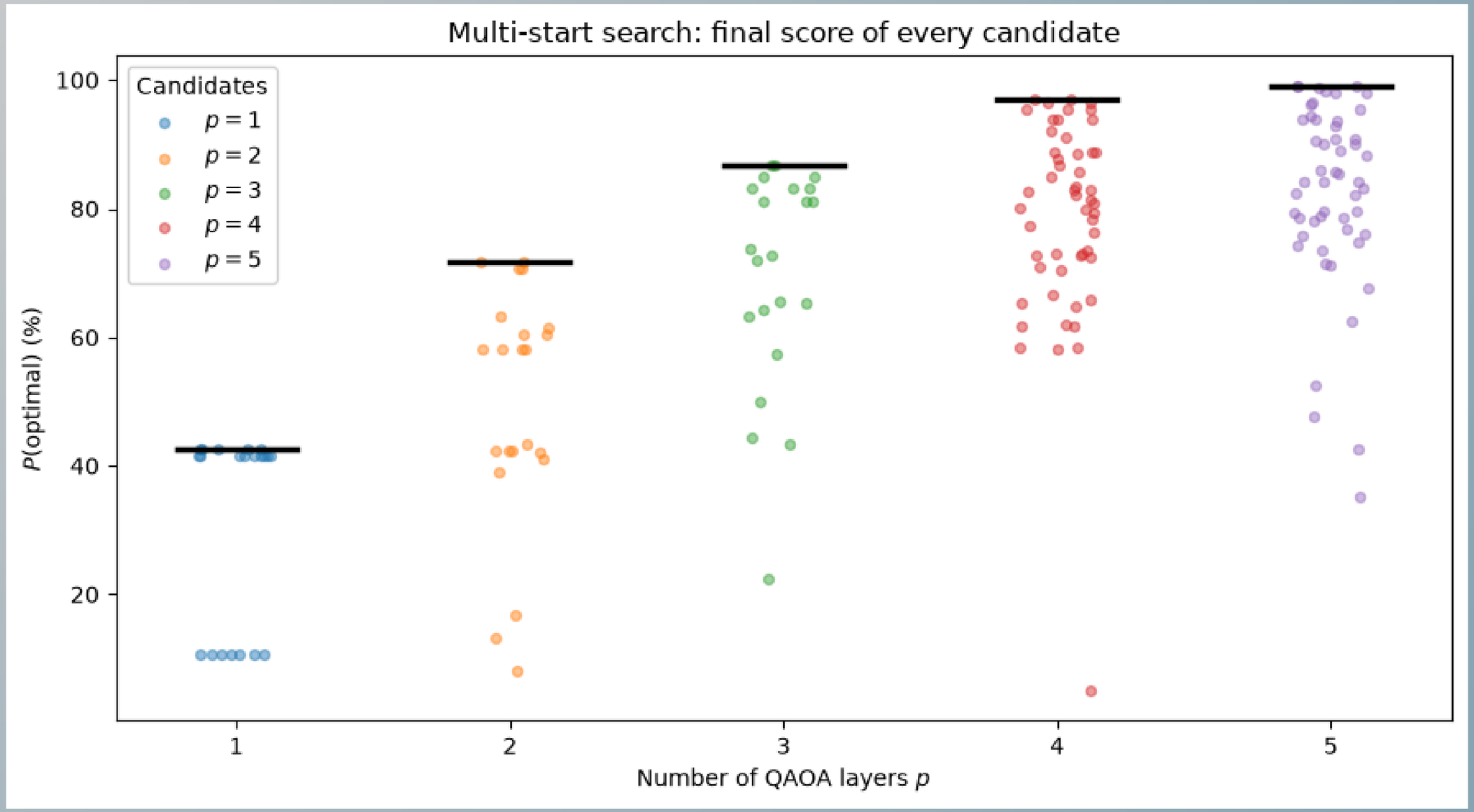

q_0
q_1
q_2
q_3
q_4
H
H
H
H
H
0
1
2
3
4
exp(-it (IIIZZ + IIZZI + IZZII + IZIIZ + ZIIIZ + ZIIZI))
γ[0]
0
1
2
3
4
exp(-it (XIIII + IXIII + IIXII + IIIXI + IIIIX))
β[0]

Problem 1

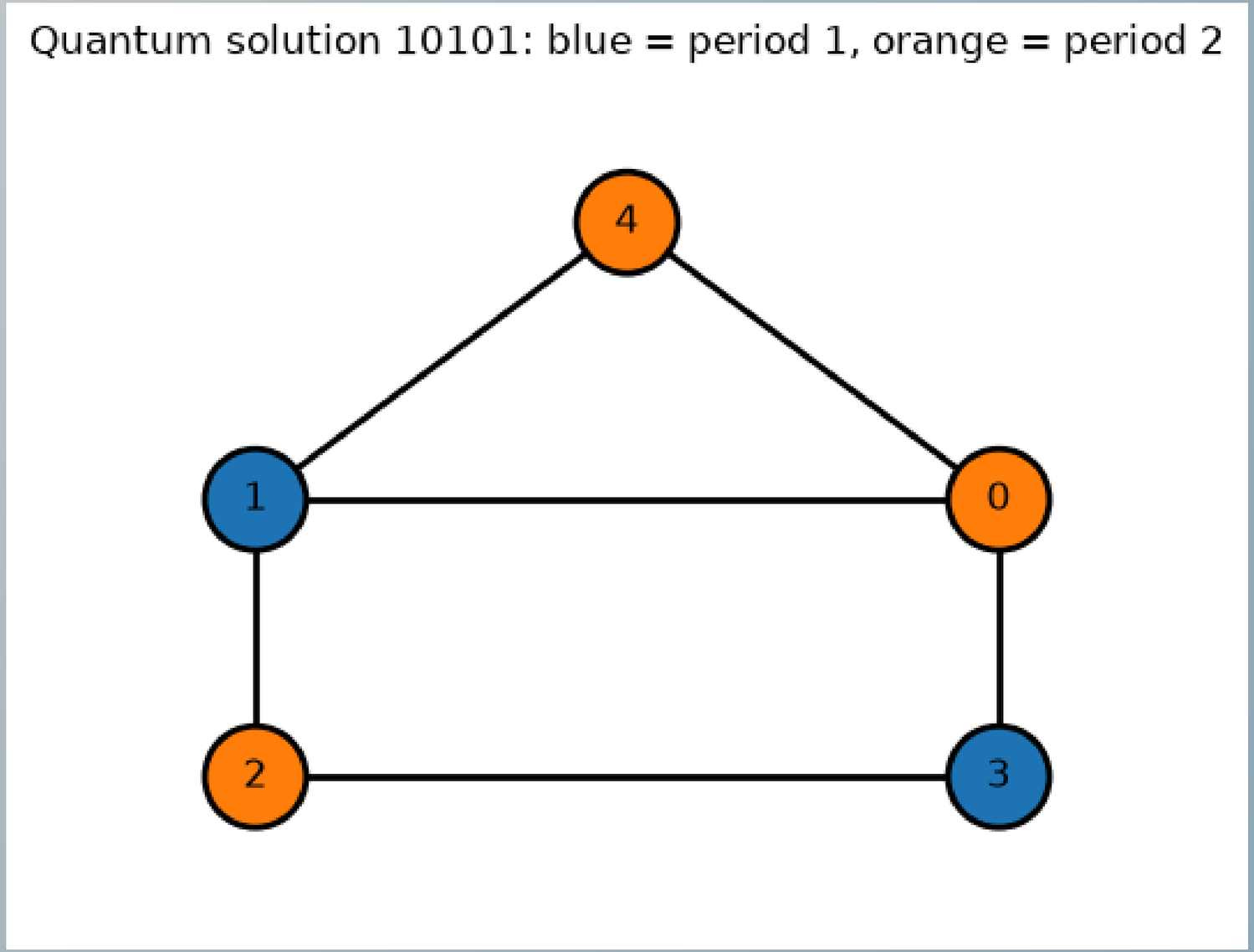

p = 1 QAOA landscape surface
P(optimal) (%)
β
γ
P(optimal) (%)


p = 1 QAOA landscape: success probability P(optimal)
best p=1 point (42.5%)
zero init (stationary)
β
γ
P(optimal) (%)

Problem 1

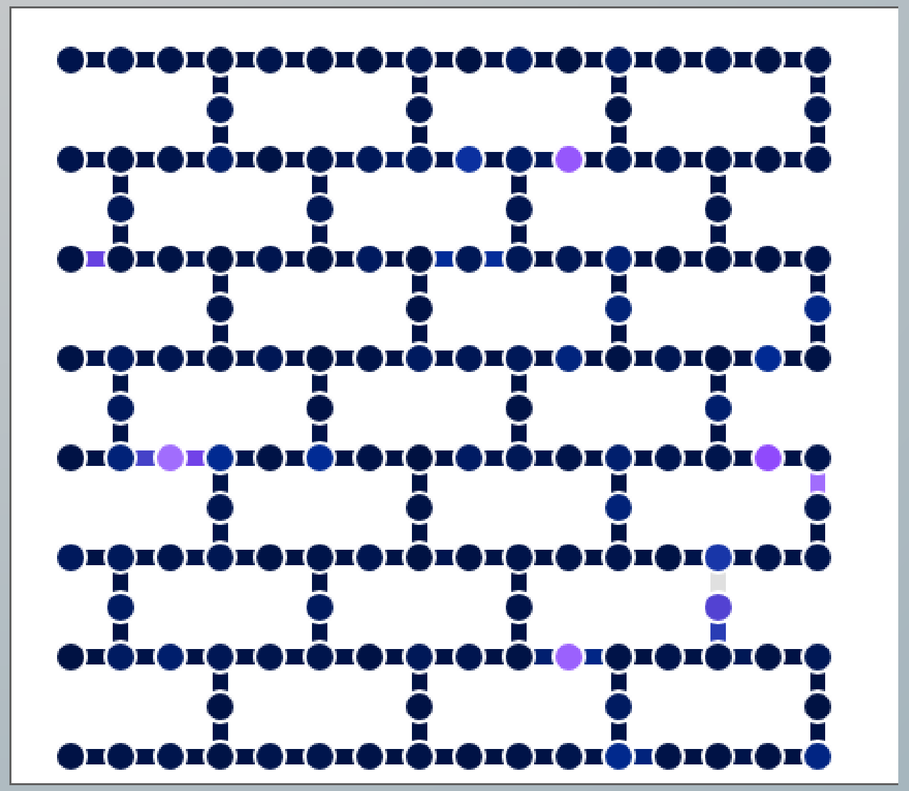

Multi-start search: final score of every candidate
Candidates
$p = 1$
$p = 2$
$p = 3$
$p = 4$
$p = 5$
100
80
60
40
20
P(optimal) (%)
1
2
3
4
5
Number of QAOA layers p

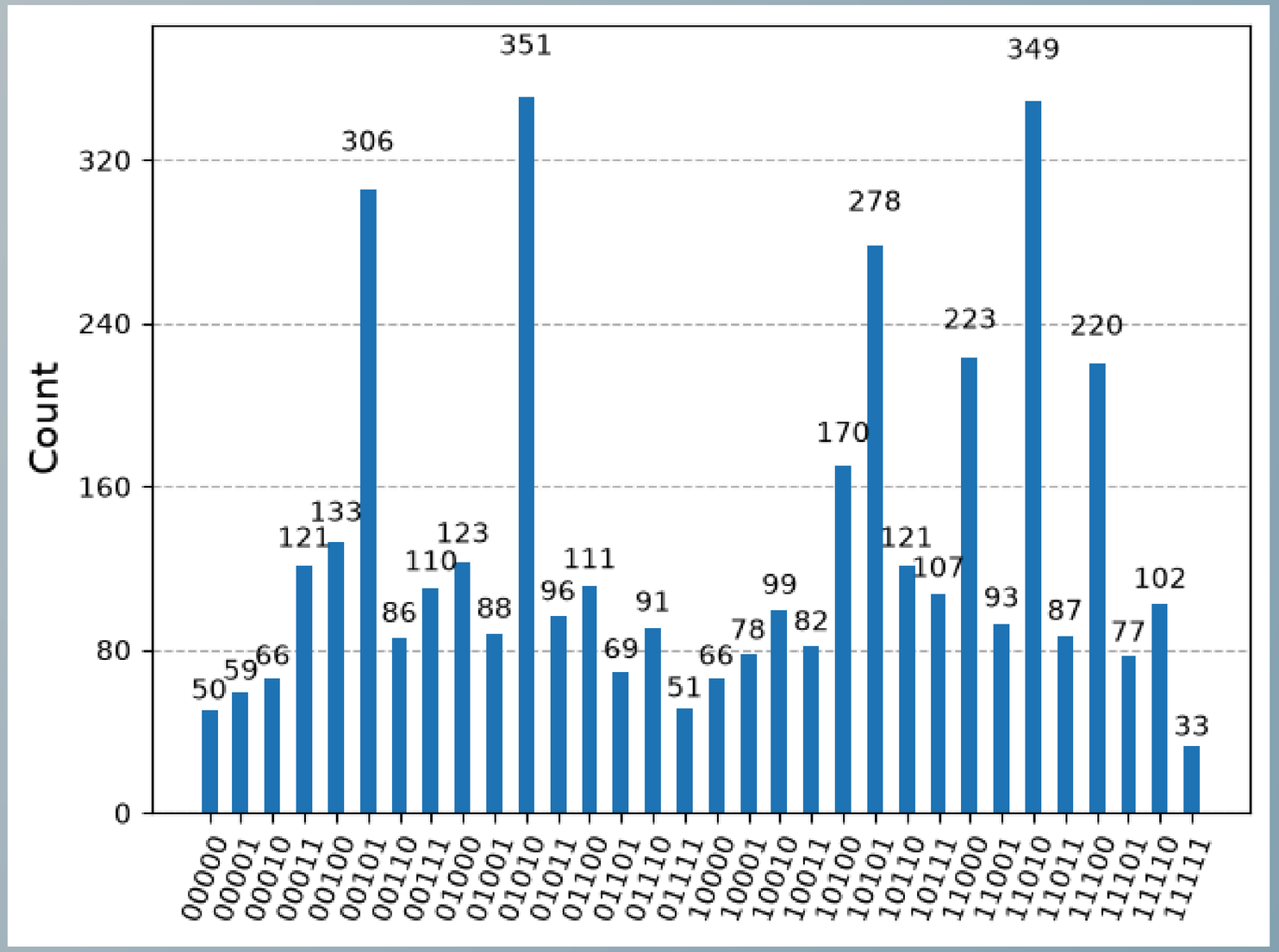
Problem 1 : One of the possible Solutions



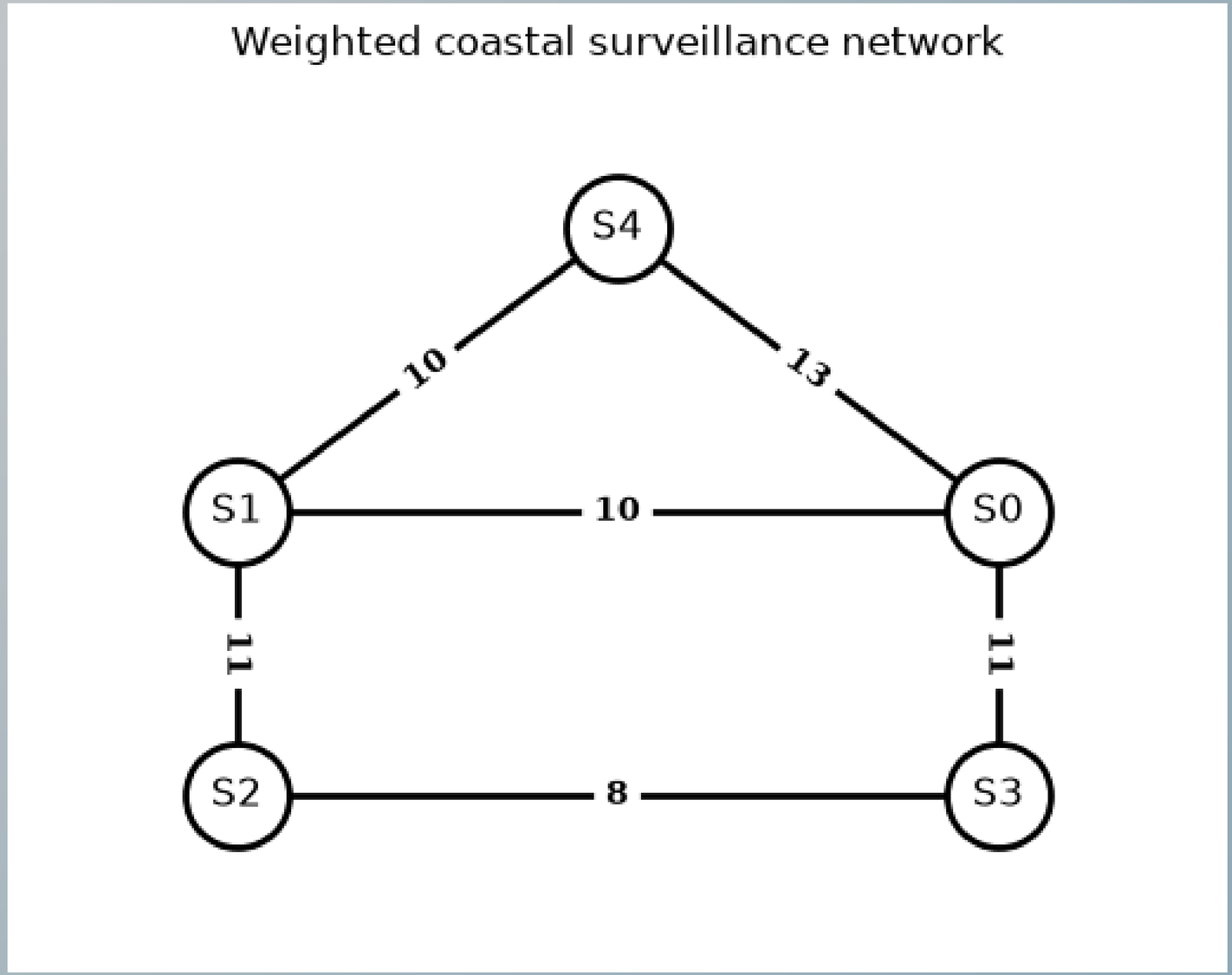
Problem 1

ibm_marrakesh





Problem 2

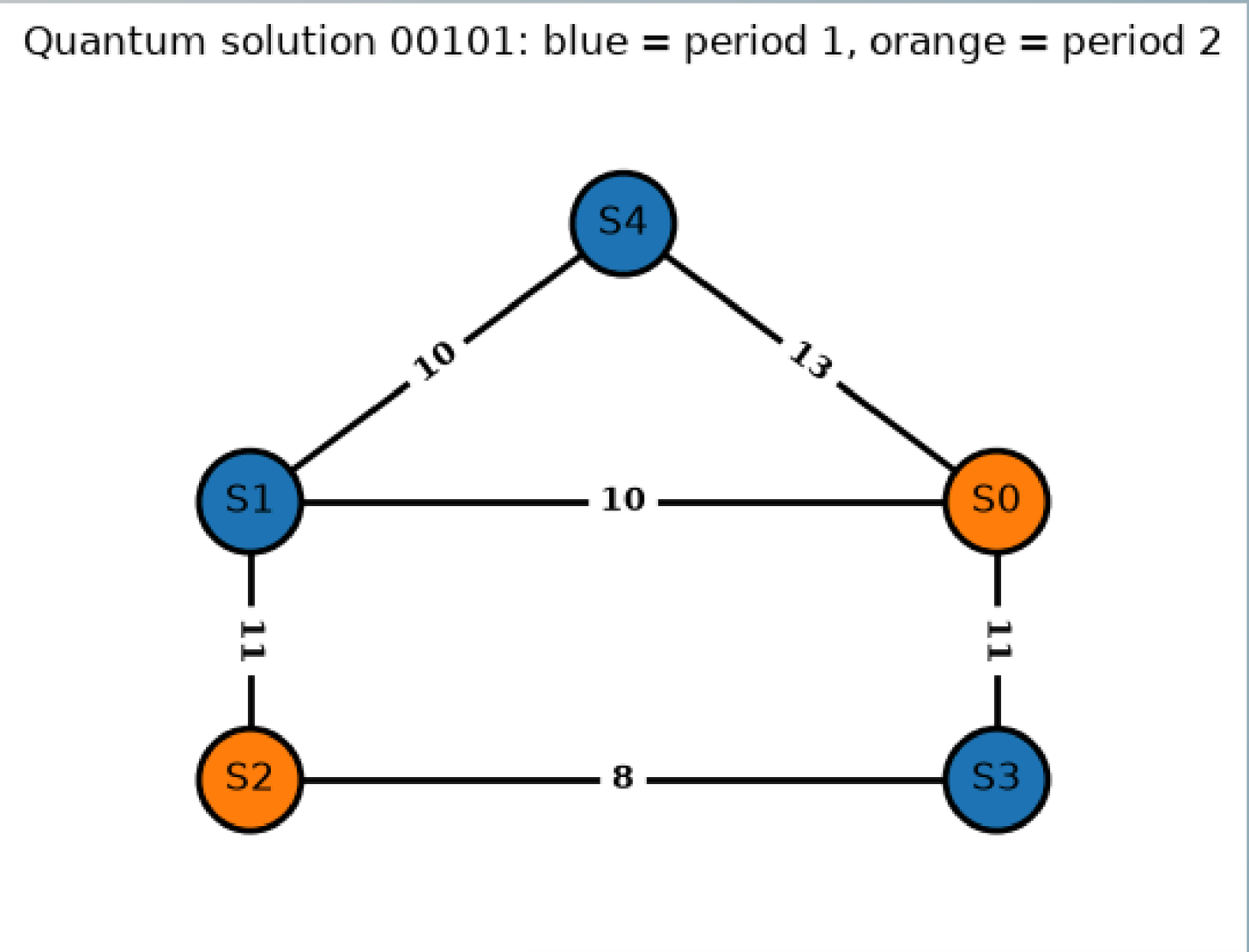

Weighted coastal surveillance network
S4
S1
S0
S2
S3
10
13
10
11
11
8

Problem 2

```python
# Normalize the weights by the largest overlap weight
max_weight = max(weight for _, _, weight in G.edges(data="weight"))

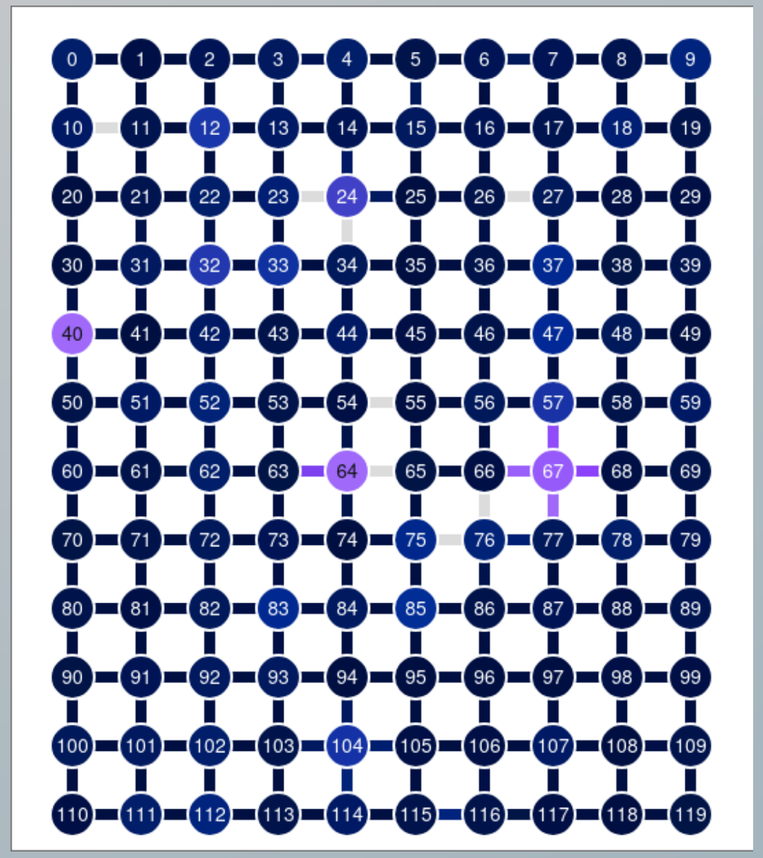
# Instantiate the SparsePauliOp as a sum of Pauli chains multiplied by a certain coefficient
H = SparsePauliOp.from_list([
    ("IIIZZ", 10.0 / max_weight),  # edge (0, 1)
    ("IIZZI", 11.0 / max_weight),  # edge (1, 2)
    ("IZZII", 8.0 / max_weight),   # edge (2, 3)
    ("IZIIZ", 11.0 / max_weight),  # edge (0, 3)
    ("ZIIIZ", 13.0 / max_weight),  # edge (0, 4)
    ("ZIIZI", 10.0 / max_weight),  # edge (1, 4)
])
```

Problem 2: One of the possible Solutions



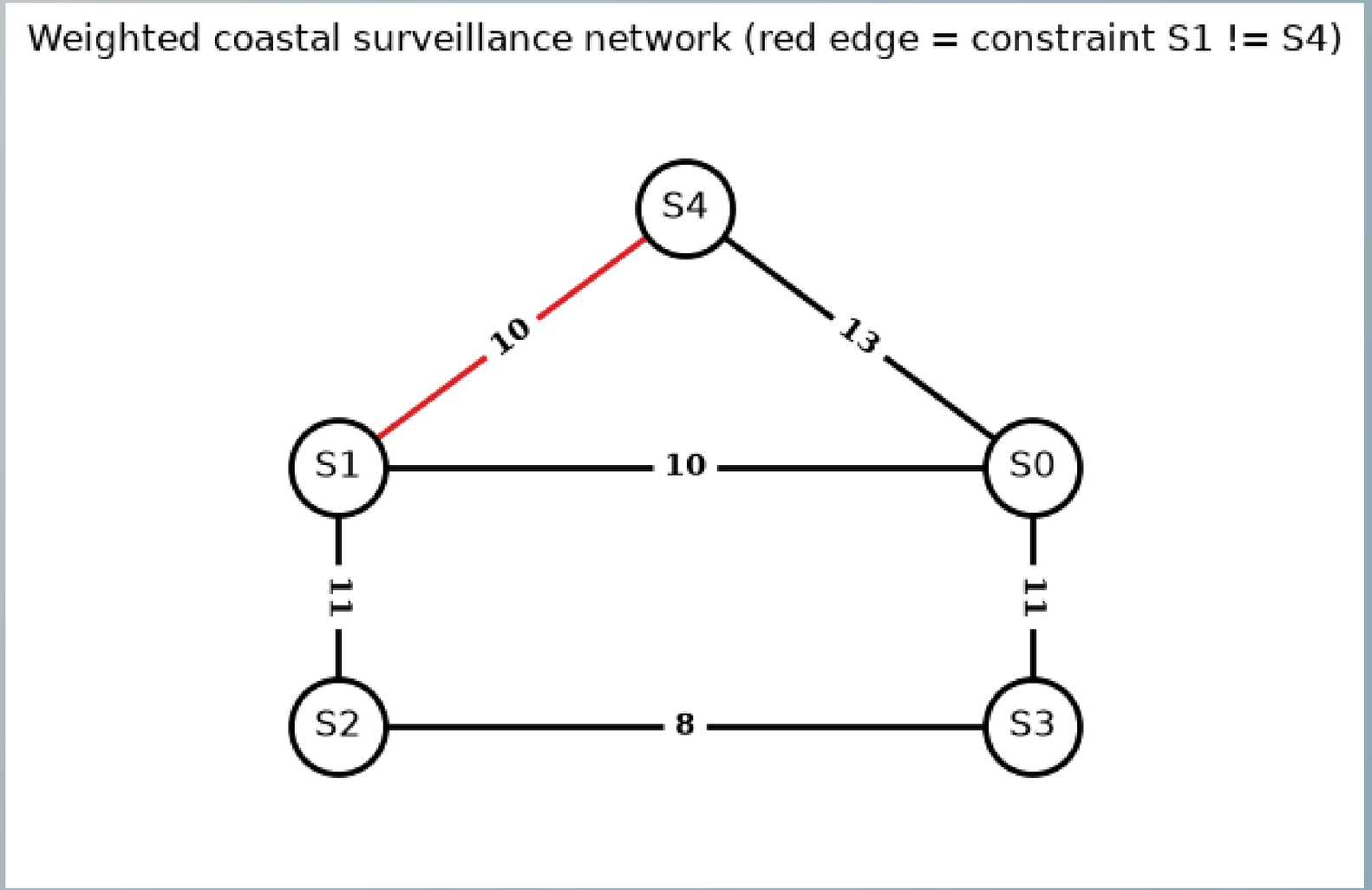
Problem 2

ibm_chicago




```
QPU backend    : ibm_chicago (120 qubits)
Pending jobs   : 0
Transpiled depth: 206
QPU job id     : dbqmc26jjaos73ahi4fg
QPU shots      : 4096
QPU score      : 68.07% of shots on an optimal solution
QPU top outcomes:
  00101  probability= 34.81%  cost=-43
  11010  probability= 33.25%  cost=-43
  01010  probability=  5.18%  cost=-37
  10101  probability=  4.03%  cost=-37
  00111  probability=  2.08%  cost=-21
```

Problem 3


Weighted coastal surveillance network (red edge = constraint S1 != S4)
S4
S1
S0
S2
S3
10
13
10
11
11
8

Problem 3

```python
# Constraint S1 != S4: (1 + Z1 Z4) / 2
constraint = SparsePauliOp.from_list([
    ("IIIII", 0.5),
    ("ZIIZI", 0.5),  # Z on qubits 4 and 1
])

# Check how the penalty weight affects the exact optimum.
for candidate_alpha in (2.0, 4.0, 6.0):
    _, candidate_sols = utils.compute_exact_sols(
        hamiltonian=H_base + candidate_alpha * constraint
    )
    feasible = all(int(sol[-2]) != int(sol[-5]) for sol in candidate_sols)
    print(
        f"alpha={candidate_alpha:4.1f} | exact solutions={candidate_sols} | "
        f"S1 != S4 satisfied: {feasible}"
    )

# Penalty weight chosen slightly above the normalized sum of weights (63/13).
alpha = 6.0

# Constrained Hamiltonian used by the rest of the notebook.
H = H_base + alpha * constraint
```

Problem 4

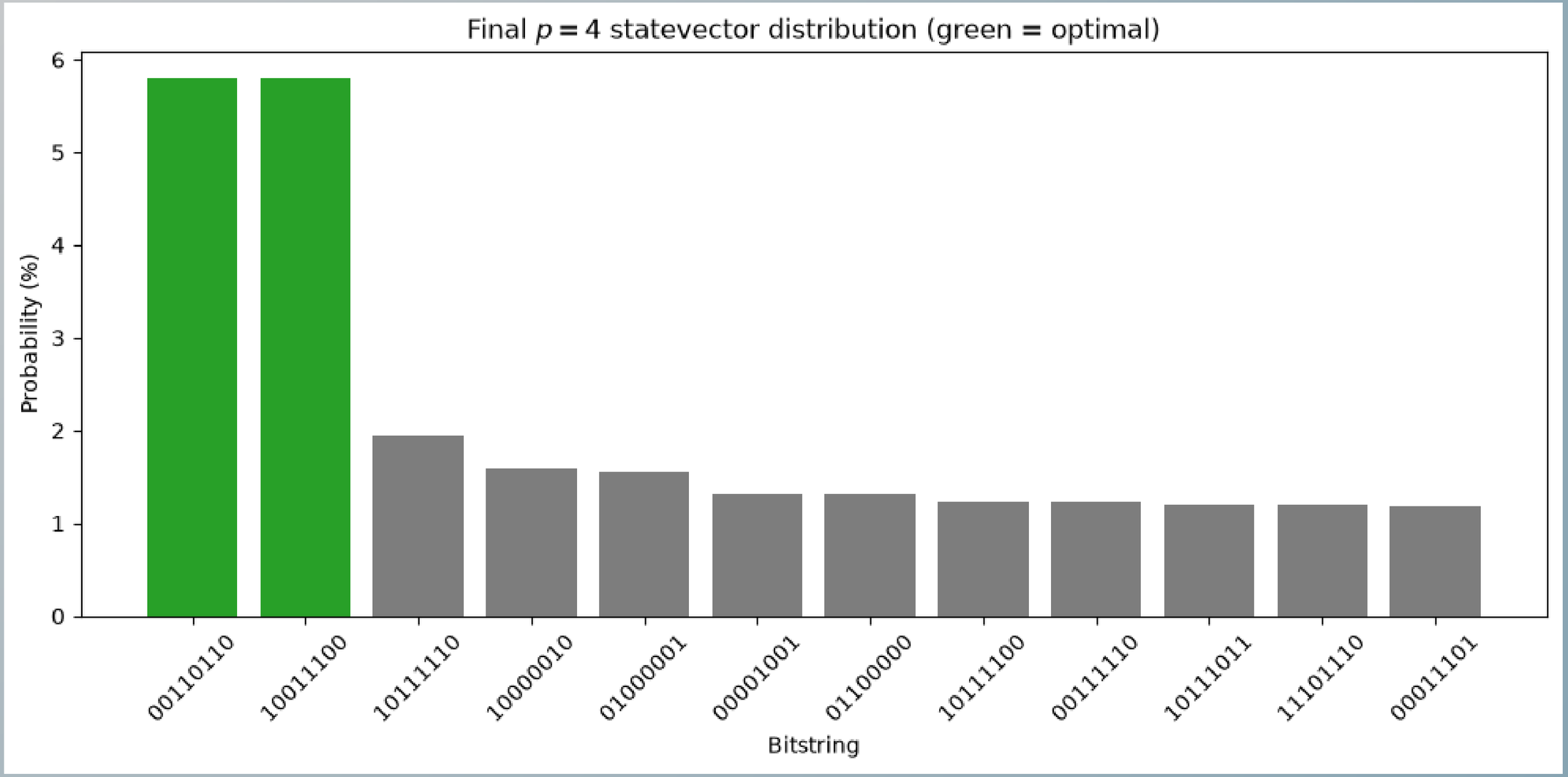
Let $P_p(s)$ be the projector onto the state "station s is visited at step p". On the two qubits of step p, it is written as

$$P_p(s) = \frac{1}{4}\left(I + (-1)^{b_0(s)} Z_{2p}\right)\left(I + (-1)^{b_1(s)} Z_{2p+1}\right),$$

where $(b_0(s), b_1(s))$ are the two bits of the station label. The unconstrained cost Hamiltonian is

$$H_{\text{cost}} = \sum_{p=0}^{2} \sum_{s,s'=0}^{3} D_{ss'}\, P_p(s)\, P_{p+1}(s'),$$

Problem 4


Final $p = 4$ statevector distribution (green = optimal)
Probability (%)
0
1
2
3
4
5
6
00110110
10011100
10111110
10000010
01000001
00001001
01100000
10111100
00111110
10111011
11101110
00011101
Bitstring

Problem 4

```
QPU backend    : ibm_boston (156 qubits)
Pending jobs   : 2
Transpiled depth: 2144
QPU job id     : dbqp0evv5hds73dgf5cg
QPU shots      : 4096
QPU score      : 0.85% of shots on an optimal route
```

Leassons learned

- QAOA
- Quantum basics
- Problem solving
- Collaboration

